Desafio Arquiteto Banco Carrefour 2
Contexto do Problema 4
Requisitos 6
Registro do Raciocínio Arquitetural 9
Decisões Arquiteturais (ADR) 14
Arquitetura da Solução 21
Modelagem de Dados 28
Estratégia de Escalabilidade 30
Fluxos e Responsabilidades da Solução 33
Arquitetura Técnica 35
Tecnologias e Ferramentas 40
Segurança 44
Observabilidade 46
Custos(FinOps) 49

Desafio Arquiteto Banco Carrefour

Description

In a sentence or two, describe the purpose of this space.

Project Tracker

Recently updated content

This list below will automatically update each time somebody in your space creates or updates content.

- Contexto do Problema
  Mar 20, 2026 • contributed by carlos Roberto Bueno
- Requisitos
  Mar 20, 2026 • contributed by carlos Roberto Bueno
- Registro do Raciocínio Arquitetural
  Mar 20, 2026 • contributed by carlos Roberto Bueno
- Decisões Arquiteturais (ADR)
  Mar 20, 2026 • contributed by carlos Roberto Bueno
- Arquitetura da Solução
  Mar 20, 2026 • contributed by carlos Roberto Bueno
- Modelagem de Dados
  Mar 20, 2026 • contributed by carlos Roberto Bueno
- Estratégia de Escalabilidade
  Mar 20, 2026 • contributed by carlos Roberto Bueno
- Fluxos e Responsabilidades da Solução
  Mar 20, 2026 • contributed by carlos Roberto Bueno
- Arquitetura Técnica
  Mar 20, 2026 • contributed by carlos Roberto Bueno
- Tecnologias e Ferramentas
  Mar 20, 2026 • contributed by carlos Roberto Bueno

Contributors

This list below will automatically update each time somebody in your space creates or updates content.

Contexto do Problema

1. Contexto do Problema

Um comerciante necessita de uma solução para gerenciar seu fluxo de caixa diário, permitindo o registro de lançamentos financeiros e a consulta do saldo consolidado por dia.

Os lançamentos representam entradas e saídas financeiras, classificadas como créditos e débitos, e devem ser registrados de forma confiável, garantindo rastreabilidade e consistência das informações ao longo do tempo.

Além do registro das transações, o sistema deve disponibilizar um mecanismo eficiente para consulta do saldo diário consolidado, possibilitando ao comerciante acompanhar sua posição financeira de forma rápida e precisa.

1.1 Objetivo da Solução

A solução proposta tem como objetivo:

- Permitir o registro de lançamentos financeiros (débitos e créditos)
- Garantir a persistência confiável dos dados
- Disponibilizar o saldo diário consolidado de forma eficiente
- Assegurar a disponibilidade do serviço de lançamentos independentemente do processo de consolidação

---

1.2 Escopo

O sistema contempla:

- Registro de lançamentos financeiros
- Consolidação de saldo diário
- Consulta de saldo consolidado

Não fazem parte do escopo:

- Validação de limites financeiros
- Processos de autorização de pagamento
- Regras complexas de negócio (ex.: saldo negativo, crédito, antifraude)

1.3 Premissas

Para a construção da solução, foram consideradas as seguintes premissas:

- Os lançamentos financeiros são imutáveis após sua criação
- O saldo é derivado a partir dos lançamentos registrados
- O sistema pode operar com consistência eventual para o saldo consolidado
- O volume de dados crescerá continuamente ao longo do tempo

1.4 Critérios de Qualidade Relevantes

Os principais atributos de qualidade considerados são:

- **Disponibilidade:** o serviço de registro de lançamentos não deve ser impactado por falhas no processo de consolidação
- **Desempenho:** o sistema deve suportar consultas frequentes de saldo diário
- **Escalabilidade:** a solução deve suportar crescimento contínuo do volume de transações
- **Manutenibilidade:** a arquitetura deve permitir evolução futura com baixo acoplamento

2. Requisitos

2.1 Requisitos Funcionais

Os requisitos funcionais descrevem as capacidades que o sistema deve oferecer para atender às necessidades do negócio.

RF-01 — Registrar Lançamento

O sistema deve permitir o registro de lançamentos financeiros, classificados como:

- Débito (saída de valor)
- Crédito (entrada de valor)

Cada lançamento deve conter, no mínimo:

- Identificador único
- Identificação do comerciante (tenant)
- Tipo (débito/crédito)
- Valor
- Data/hora de ocorrência

RF-02 — Consultar Lançamentos

O sistema deve permitir a consulta dos lançamentos financeiros com base em:

- Comerciante (tenant)
- Intervalo de datas

RF-03 — Consolidar Saldo Diário

O sistema deve ser capaz de consolidar os lançamentos financeiros por dia, calculando:

- Total de créditos
- Total de débitos
- Saldo final do dia

RF-04 — Consultar Saldo Diário

O sistema deve permitir a consulta do saldo consolidado por dia, considerando:

- Comerciante (tenant)
- Data específica

2.2 Requisitos Não Funcionais

Os requisitos não funcionais definem as características de qualidade da solução.

RNF-01 — Disponibilidade

O serviço de registro de lançamentos deve permanecer disponível mesmo em caso de falha no processo de consolidação de saldo.

RNF-02 — Desempenho

O serviço de consolidação e consulta de saldo deve suportar:

- Até 50 requisições por segundo
- Com tolerância máxima de 5% de perda de requisições

RNF-03 — Escalabilidade

A solução deve ser capaz de suportar crescimento contínuo do volume de lançamentos sem degradação significativa de desempenho.

RNF-04 — Consistência

O sistema pode adotar consistência eventual para o saldo consolidado, desde que:

- Os lançamentos sejam persistidos de forma confiável
- O saldo seja eventualmente refletido corretamente

RNF-05 — Isolamento de Dados

Os dados dos comerciantes devem ser isolados logicamente, garantindo que:

- Um comerciante não tenha acesso aos dados de outro
- Consultas sejam sempre filtradas por identificador do tenant

---

RNF-06 — Auditabilidade

O sistema deve permitir rastreabilidade completa dos lançamentos financeiros, garantindo histórico íntegro das operações realizadas.

2.3 Premissas Técnicas

Para a definição da solução, foram consideradas as seguintes premissas técnicas:

- A solução poderá ser implementada em ambiente local para fins de execução e validação
- Não há restrição quanto à tecnologia de persistência (relacional ou NoSQL)
- A arquitetura deve ser independente da tecnologia específica, priorizando conceitos arquiteturais
- A comunicação entre componentes pode ser síncrona ou assíncrona, conforme decisão arquitetural

3. Registro do Raciocínio Arquitetural

3.1 Abordagem de Análise

A análise da solução foi conduzida a partir da compreensão do domínio de negócio e da identificação dos principais problemas plausíveis dentro do escopo definido.

A partir disso, foram levantadas perguntas arquiteturais com os seguintes objetivos:

- identificar riscos
- evitar complexidade desnecessária
- alinhar decisões ao contexto do problema

A abordagem adotada seguiu o fluxo:

- compreensão do domínio
- identificação de problemas
- levantamento de hipóteses
- análise de trade-offs
- tomada de decisão

3.2 Análise do Modelo de Dados

Pergunta
Os registros financeiros devem ser mutáveis ou imutáveis?

Análise
Sistemas financeiros tradicionalmente utilizam modelos baseados em registros imutáveis para garantir:

- rastreabilidade
- auditabilidade
- capacidade de reprocessamento

A atualização de registros financeiros pode introduzir inconsistências e dificultar auditorias.

Conclusão
Os lançamentos financeiros serão tratados como registros imutáveis (append-only), compondo

um modelo de ledger.

3.3 Análise de Dependência entre Serviços

Pergunta
O serviço de consolidação pode impactar o serviço de registro de lançamentos?

Análise
Uma integração síncrona entre os serviços criaria acoplamento direto, podendo causar indisponibilidade em cascata.

Considerando o requisito de disponibilidade, o serviço de lançamentos deve operar de forma independente.

Conclusão
A comunicação entre os serviços será realizada de forma assíncrona, utilizando eventos para promover desacoplamento.

3.4 Análise de Cálculo de Saldo

Pergunta
O saldo deve ser calculado sob demanda ou pré-processado?

Análise
O cálculo do saldo a partir da agregação de transações pode se tornar custoso com o crescimento da base de dados.

Consultas frequentes sobre grandes volumes tendem a impactar negativamente o desempenho.

Conclusão
O saldo será mantido em um modelo de leitura otimizado (read model), atualizado a partir dos eventos de transação.

3.5 Análise de Ordenação de Eventos

Pergunta
A ordem entre lançamentos impacta o resultado do saldo?

Análise
O saldo consolidado é derivado da soma dos lançamentos financeiros.

Para operações independentes (crédito e débito), a ordem de processamento não altera o resultado final.

Não foram identificadas regras de negócio que exijam ordenação estrita entre eventos.

Conclusão
Não é necessário garantir ordenação forte entre eventos, simplificando a arquitetura e evitando mecanismos complexos de coordenação.

3.6 Análise de Crescimento do Ledger

Pergunta
Como lidar com o crescimento contínuo da base de transações?

Análise
Mesmo com a utilização de um modelo de leitura otimizado, o volume de transações crescerá continuamente.

Esse crescimento pode impactar:

- tamanho dos índices
- custo de manutenção
- desempenho de consultas históricas

Conclusão
Será adotado particionamento temporal do ledger, permitindo:

- melhor gerenciamento dos dados
- redução do tamanho de índices por partição
- facilidade de arquivamento

3.7 Análise de Estratégia de Armazenamento

Pergunta
Como otimizar o acesso a dados recentes sem comprometer o histórico?

Análise
Os dados mais recentes possuem maior frequência de acesso, enquanto dados históricos são menos acessados.

Conclusão
A solução poderá evoluir para uma estratégia de armazenamento em camadas (hot/cold),

mantendo dados recentes em armazenamento otimizado e arquivando dados históricos.

3.8 Análise de Consistência

Pergunta
O sistema exige consistência imediata para o saldo consolidado?

Análise
O saldo é derivado a partir de eventos de transação e pode ser atualizado de forma assíncrona.

O domínio não apresenta exigência de consistência forte imediata.

Conclusão
Será adotado modelo de consistência eventual para o saldo consolidado.

3.9 Análise de Isolamento de Dados

Pergunta
Como garantir isolamento entre comerciantes?

Análise
Os dados devem ser segregados logicamente para evitar acesso indevido entre diferentes comerciantes.

Conclusão
Todos os dados serão associados a um identificador de tenant, garantindo isolamento lógico.

3.10 Análise do Papel da Fila e Persistência

Pergunta
A fila de eventos pode substituir o ledger como mecanismo de armazenamento?

Análise
A fila atua como mecanismo de transporte de eventos, não garantindo persistência de longo prazo, capacidade de consulta ou reprocessamento confiável.

Conclusão
A fila será utilizada exclusivamente para propagação de eventos, enquanto o ledger será responsável pelo armazenamento permanente e confiável dos dados.

3.11 Análise da Estratégia de Persistência

Pergunta
A API deve persistir os dados ou apenas publicar eventos?

Análise
Foi considerada a abordagem event-first, na qual a API apenas publica eventos e um serviço independente realiza a persistência.

No entanto, essa abordagem introduz maior complexidade operacional e desafios de consistência.

Conclusão
A Transaction API será responsável pela persistência no ledger e publicação de eventos, garantindo simplicidade e consistência no registro inicial.

3.12 Síntese das Decisões

A partir da análise realizada, foram definidas as seguintes diretrizes arquiteturais:

- adoção de modelo de registro imutável (append-only) para os lançamentos financeiros
- utilização de integração assíncrona orientada a eventos para desacoplamento entre componentes
- separação entre modelo de escrita e modelo de leitura (read model)
- organização temporal dos dados para suportar crescimento contínuo
- adoção de consistência eventual para o saldo consolidado
- garantia de isolamento lógico por identificador de tenant
- utilização do ledger como fonte de verdade e da fila como mecanismo de propagação

---

3.13 Consideração sobre o Modelo de Leitura

A solução adota o conceito de modelo de leitura (read model), permitindo que os dados sejam estruturados de forma otimizada para consultas.

A implementação desse modelo é independente da tecnologia de persistência, podendo ser realizada em bancos relacionais, NoSQL ou orientados a análise, conforme o contexto de implantação.

Decisões Arquiteturais (ADR)

4. Decisões Arquiteturais (ADR)

Você pode colocar essa introdução:

---

4. Decisões Arquiteturais (ADR)

Esta seção apresenta as principais decisões arquiteturais adotadas na solução, bem como suas motivações e impactos.

Cada decisão foi tomada com base na análise do domínio, requisitos e critérios de qualidade definidos.

ADR-001 — Modelo de Registro Imutável (Ledger Append-Only)

Status: Aprovado

Contexto
O sistema precisa registrar lançamentos financeiros garantindo rastreabilidade e integridade dos dados ao longo do tempo.

Decisão
Adotar modelo de registro imutável (append-only) para os lançamentos financeiros.

Alternativas consideradas

- Atualização de registros existentes
- Manutenção de saldo diretamente no banco

Justificativa
O modelo append-only garante histórico completo das operações, facilitando auditoria e evitando inconsistências decorrentes de atualizações concorrentes.

Consequências

- (+) Alta auditabilidade
- (+) Facilidade de reprocessamento
- (-) Crescimento contínuo da base de dados

ADR-002 — Integração Assíncrona Orientada a Eventos

Status: Aprovado

Contexto
O serviço de registro de lançamentos não deve ser impactado por falhas no processo de consolidação.

Decisão
Adotar integração assíncrona orientada a eventos entre os serviços.

Alternativas consideradas

- Comunicação síncrona entre serviços
- Processamento acoplado no mesmo serviço

Justificativa
A comunicação assíncrona reduz o acoplamento entre componentes e aumenta a resiliência do sistema.

Consequências

- (+) Maior disponibilidade
- (+) Desacoplamento entre serviços
- (-) Introdução de consistência eventual

ADR-003 — Separação entre Modelo de Escrita e Leitura (Read Model)

Status: Aprovado

Contexto
Consultas frequentes de saldo podem se tornar custosas quando baseadas diretamente no ledger de transações.

Decisão
Separar o modelo de escrita do modelo de leitura, utilizando um read model otimizado para consultas.

Alternativas consideradas

- Cálculo de saldo sob demanda
- Uso direto do ledger para consultas

Justificativa
A separação permite otimizar consultas sem comprometer o modelo de escrita, além de manter independência tecnológica.

Consequências

- (+) Melhor desempenho de leitura
- (+) Flexibilidade na escolha da tecnologia
- (-) Necessidade de sincronização entre modelos

ADR-004 — Organização Temporal dos Dados

Status: Aprovado

Contexto
O volume de transações tende a crescer continuamente ao longo do tempo.

Decisão
Adotar estratégia de organização temporal dos dados para suportar crescimento do ledger.

Alternativas consideradas

- Tabela única sem segmentação
- Estratégias de arquivamento manual

Justificativa
A organização temporal permite melhor gerenciamento de dados, otimização de consultas por período e facilita estratégias de arquivamento.

Consequências

- (+) Melhor desempenho em consultas temporais
- (+) Facilidade de arquivamento
- (-) Maior complexidade na gestão de dados

ADR-005 — Consistência Eventual para Consolidação

Status: Aprovado

Contexto
O saldo consolidado é derivado a partir dos lançamentos e não precisa refletir imediatamente todas as transações.

Decisão
Adotar consistência eventual para o cálculo do saldo consolidado.

Alternativas consideradas

- Consistência forte com processamento síncrono
- Atualização imediata do saldo

Justificativa
A consistência eventual permite maior escalabilidade e desacoplamento entre componentes, atendendo aos requisitos do domínio.

Consequências

- (+) Melhor escalabilidade
- (+) Redução de acoplamento
- (-) Possível atraso na atualização do saldo

ADR-006 — Isolamento Lógico por Tenant

Status: Aprovado

Contexto
O sistema deve garantir que os dados de um comerciante não sejam acessíveis por outros.

Decisão
Adotar isolamento lógico de dados baseado em identificador de tenant.

Alternativas consideradas

- Base de dados única sem segregação
- Isolamento físico por cliente

Justificativa
O isolamento lógico permite escalabilidade e simplicidade operacional, mantendo segurança adequada para o contexto.

Consequências

- (+) Simplicidade de implementação
- (+) Escalabilidade
- (-) Necessidade de controle rigoroso nas consultas

ADR-007 — Estratégia de Persistência no Fluxo de Escrita

Contexto

Foi considerada a adoção de um modelo totalmente orientado a eventos, no qual a API apenas publicaria eventos e um serviço independente seria responsável pela persistência.

Decisão

A Transaction API será responsável por persistir os dados no ledger e publicar eventos.

Justificativa

A abordagem reduz a complexidade da solução, garante confirmação imediata da persistência e evita dependência de processamento assíncrono para consistência inicial.

Consequências

- (+) Consistência imediata no registro
- (+) Simplicidade arquitetural
- (-) Maior acoplamento entre API e persistência

ADR-008 — Separação entre Write Model e Read Model (CQRS)

Contexto

Necessidade de otimizar operações de escrita e leitura com características distintas.

Decisão

Separar o modelo de escrita (ledger) do modelo de leitura (read model).

Justificativa

Permite otimizar leitura e escrita de forma independente, reduzindo impacto de consultas sobre operações críticas.

Consequências

- (+) Melhor performance
- (+) Escalabilidade independente
- (-) Consistência eventual
- (-) Maior complexidade

ADR-009 — Uso de Arquitetura Orientada a Eventos

Contexto

Necessidade de desacoplar o registro de transações do processamento do saldo.

Decisão

Utilizar eventos para comunicação entre serviços.

Justificativa

Permite processamento assíncrono, maior escalabilidade e desacoplamento.

ADR-010 — Ledger como Fonte de Verdade

Contexto

Necessidade de garantir rastreabilidade e integridade dos dados financeiros.

Decisão

Adotar modelo append-only (ledger) como fonte de verdade.

Justificativa

Permite auditoria, reprocessamento e reconstrução de estado.

ADR-011 — Estratégia de Reprocessamento

Contexto

Possibilidade de inconsistências no modelo de leitura.

Decisão

Permitir reconstrução do read model a partir do ledger.

Justificativa

Garante recuperação em cenários de falha ou mudança de regra.

ADR-012 — Uso de Dead Letter Queue (DLQ)

Contexto

Eventos podem falhar durante o processamento.

Decisão

Utilizar DLQ para isolamento de falhas.

Justificativa

Evita bloqueio do fluxo principal e permite reprocessamento pontual.

ADR-013 — Uso de API Gateway

Contexto

Foi considerada a utilização de API Gateway para centralização de controle de entrada.

Decisão

Optou-se por não utilizar API Gateway nesta solução.

Justificativa

Dado o escopo reduzido da aplicação e o número limitado de serviços expostos, as responsabilidades de autenticação e controle de acesso podem ser tratadas diretamente nas APIs.

Essa abordagem reduz complexidade operacional e custo, mantendo aderência aos requisitos do problema.

Consequências

- (+) Redução de complexidade
- (+) Menor custo
- (-) Menor centralização de políticas

5. Arquitetura da Solução

5.1 Visão Geral da Arquitetura

A solução proposta foi estruturada com base na separação clara de responsabilidades entre escrita, processamento e leitura, garantindo baixo acoplamento, escalabilidade e resiliência.

O fluxo principal da solução é dividido em três etapas:

- **Registro de lançamentos (Write):** responsável pela persistência dos dados financeiros
- **Processamento assíncrono:** responsável pela consolidação do saldo
- **Consulta de dados (Read):** responsável pela recuperação eficiente das informações

Essa abordagem permite que o sistema atenda aos requisitos de disponibilidade e desempenho, evitando dependências diretas entre o registro de transações e a consolidação do saldo.

A arquitetura adota integração assíncrona orientada a eventos e separação entre modelo de escrita e leitura (read model), garantindo flexibilidade na evolução da solução.

5.2 Diagramas Arquiteturais (C4)

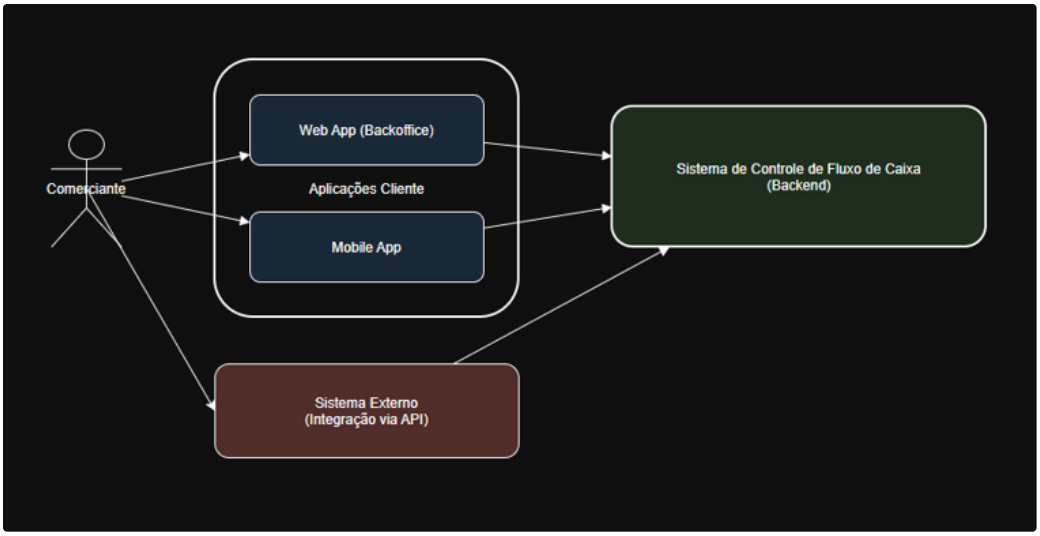
A solução é representada utilizando o modelo C4, permitindo a visualização da arquitetura em diferentes níveis de abstração.

5.2.1 Diagrama de Contexto (C4 - Nível 1)

O diagrama de contexto apresenta os atores e sistemas externos que interagem com a solução.

O sistema de controle de fluxo de caixa é consumido por aplicações cliente, como Web App e Mobile App, utilizadas pelo comerciante para registrar lançamentos e consultar saldo.

Adicionalmente, o sistema pode ser integrado a sistemas externos por meio de APIs, permitindo automação de processos e integração com outras plataformas.



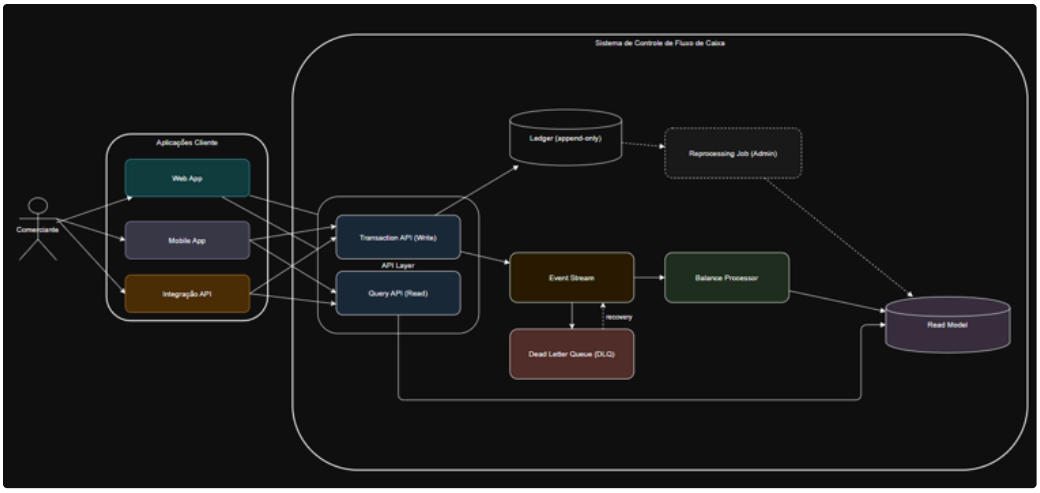
5.2.2 Diagrama de Containers (C4 - Nível 2)

O diagrama de containers apresenta os principais componentes da solução e suas responsabilidades.

A arquitetura é composta pelos seguintes elementos:

- **Aplicações Cliente:** responsáveis pela interação com o usuário final
- **Transaction API (Write):** responsável pelo registro de lançamentos financeiros
- **Query API (Read):** responsável pela consulta de saldo consolidado
- **Ledger:** responsável pelo armazenamento dos lançamentos em modelo append-only(**Fonte da verdade**)
- **Event Stream:** responsável pela comunicação assíncrona entre os componentes
- **Balance Processor:** responsável pelo processamento dos eventos e consolidação do saldo
- **Read Model:** responsável por armazenar o saldo consolidado de forma otimizada para leitura
- **Reprocessing Job:** responsável por reprocessar o saldo analisando a fonte da verdade ledger

A separação entre escrita e leitura permite melhor desempenho e escalabilidade, além de reduzir o acoplamento entre os componentes.



5.2.3 Diagrama de Componentes (C4 - Nível 3)

O diagrama de componentes detalha os serviços com maior complexidade de domínio, evidenciando a organização interna e a separação de responsabilidades.

O detalhamento foi aplicado à **Transaction API** e ao **Balance Processor**, por concentrarem a lógica de negócio da solução.

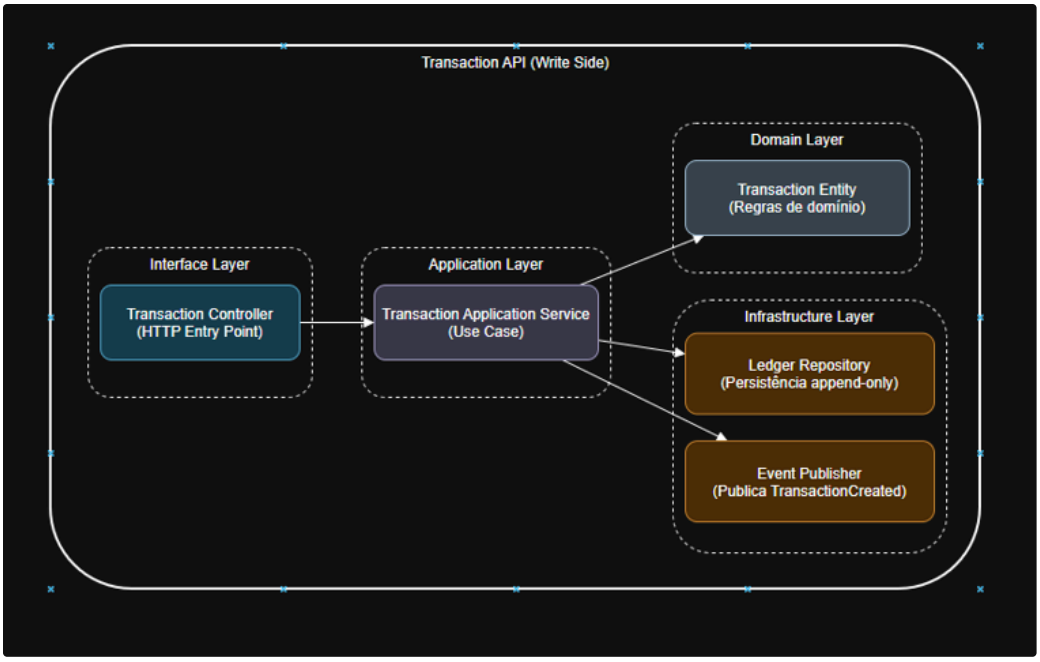
Transaction API (Write Side)

A Transaction API é responsável por receber requisições de registro de lançamentos e orquestrar o fluxo de persistência e publicação de eventos.

Sua estrutura segue princípios de separação de camadas:

- **Controller:** responsável por receber requisições HTTP
- **Application Service:** responsável por orquestrar o caso de uso
- **Domain:** responsável por encapsular as regras de negócio
- **Repository:** responsável pela persistência no ledger
- **Event Publisher:** responsável pela publicação de eventos

Essa organização permite baixo acoplamento e alta testabilidade.



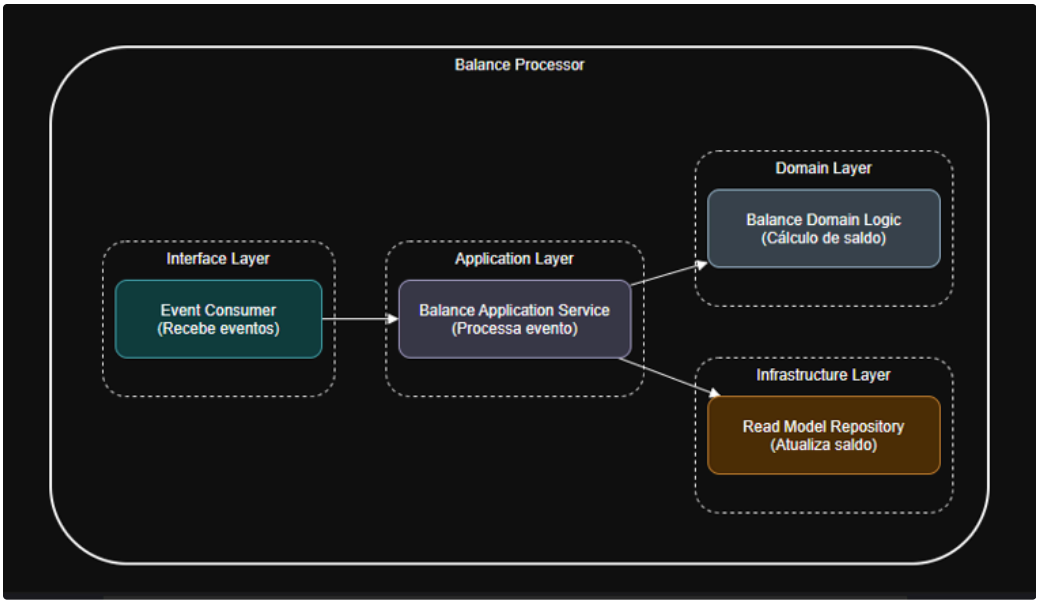
Balance Processor

O Balance Processor é responsável por consumir eventos de transação e atualizar o saldo consolidado.

Sua estrutura é composta por:

- **Event Consumer:** responsável por consumir eventos do event stream
- **Application Service:** responsável por orquestrar o processamento
- **Domain Logic:** responsável pelo cálculo do saldo
- **Repository:** responsável por atualizar o read model

Esse componente é fundamental para garantir a atualização assíncrona do saldo.



5.2.4 Papel dos Componentes

Papel dos Componentes na Arquitetura

A arquitetura da solução é composta por componentes com responsabilidades bem definidas, organizados de forma a garantir separação entre escrita, processamento e leitura.

- **Transaction API:** responsável pelo registro dos lançamentos financeiros e persistência no ledger
- **Event Stream:** responsável pela propagação assíncrona dos eventos entre os componentes
- **Balance Processor:** responsável pelo consumo dos eventos e atualização do saldo consolidado
- **Query API:** responsável pela consulta do modelo de leitura
- **Ledger:** responsável pelo armazenamento imutável dos lançamentos (fonte de verdade)
- **Read Model:** responsável pela disponibilização eficiente do saldo para consulta

---

5.2.5 Papel da Fila (Event Stream)

Papel da Fila na Arquitetura

A fila (event stream) é utilizada como mecanismo de comunicação assíncrona entre os componentes, permitindo desacoplamento e processamento distribuído.

Ela atua como um buffer temporário de eventos, garantindo que as transações sejam propagadas para os consumidores.

A fila não é utilizada como fonte de verdade, sendo o ledger responsável pelo armazenamento permanente e confiável dos dados.

5.3 Modelo de Processamento de Dados

A arquitetura estabelece uma separação clara entre armazenamento, propagação e consumo de dados:

- O **ledger** armazena os fatos financeiros de forma permanente
- Os **eventos** representam a propagação desses fatos entre os componentes
- O **modelo de leitura** materializa o estado derivado para consulta

Essa separação permite desacoplamento, escalabilidade e melhor organização das responsabilidades.

5.4 Tratamento de Falhas e DLQ

A arquitetura prevê o uso de Dead Letter Queue (DLQ) para tratamento de falhas no processamento de eventos.

Quando um evento não pode ser processado com sucesso após tentativas de retry, ele é direcionado para a DLQ, evitando bloqueio do fluxo principal.

A DLQ permite análise e reprocessamento individual de eventos.

5.5 Estratégia de Recuperação e Reprocessamento

A solução prevê a capacidade de reconstrução do modelo de leitura a partir do ledger.

Em cenários de inconsistência sistêmica ou falhas de processamento, é possível executar um processo de replay, no qual os dados do ledger são utilizados como fonte de verdade para reconstrução do saldo consolidado.

Essa estratégia complementa o uso da DLQ, permitindo tanto recuperação pontual quanto reconstrução completa.

5.6 Considerações sobre o nível de detalhamento

O detalhamento em nível de componentes foi aplicado aos elementos que concentram lógica de domínio e processamento crítico da solução, especificamente a Transaction API e o Balance Processor.

Esses componentes são responsáveis por regras de negócio, orquestração de casos de uso e transformação de dados, justificando uma decomposição mais aprofundada.

A Query API não foi detalhada neste nível por possuir responsabilidade restrita à consulta do modelo de leitura, atuando como uma camada de acesso sem lógica de domínio relevante.

Adicionalmente, componentes de natureza operacional, como Dead Letter Queue (DLQ) e o processo de reprocessamento, não foram detalhados em nível de componentes, por não fazerem parte do fluxo principal de negócio, sendo utilizados em cenários específicos de tratamento de falhas e recuperação.

O nível de detalhamento adotado segue o princípio de foco nos elementos críticos da solução, equilibrando clareza arquitetural e evitando complexidade desnecessária na representação.

Modelagem de Dados

6. Modelo de Dados

6.1 Visão Geral

A solução adota um modelo baseado em registro imutável (append-only) para os lançamentos financeiros.

6.2 Ledger de Transações

Estrutura

```
Transaction
------------
id
tenant_id
type
amount
occurred_at
created_at
```

Características

- Imutável
- Append-only
- Fonte de verdade

6.3 Modelo de Leitura

```
DailyBalance
-------------
tenant_id
date
total_credit
total_debit
balance
```

Características

- Derivado
- Mutável
- Otimizado para leitura

6.4 Indexação

- Ledger → tenant_id + occurred_at
- Read Model → tenant_id + date

6.5 Particionamento

Exemplo:

```
transactions_2026_01
transactions_2026_02
```

7. Estratégia de Escalabilidade

7.1 Visão Geral

A solução foi projetada para suportar crescimento contínuo de dados e aumento de carga, mantendo desempenho, disponibilidade e resiliência.

A estratégia de escalabilidade é baseada em:

- separação entre escrita e leitura
- processamento assíncrono orientado a eventos
- distribuição de carga entre componentes
- desacoplamento entre serviços

7.2 Escalabilidade do Registro de Lançamentos

O serviço de registro de lançamentos (Transaction API) pode ser escalado horizontalmente, permitindo múltiplas instâncias processando requisições simultaneamente.

O uso de um modelo append-only reduz conflitos de escrita e favorece alta taxa de inserção.

7.3 Escalabilidade do Processamento de Eventos

O Balance Processor pode ser escalado horizontalmente por meio de múltiplos consumidores do event stream.

Essa abordagem permite:

- aumento de throughput
- distribuição de carga
- processamento paralelo de eventos

7.4 Escalabilidade do Modelo de Leitura

O modelo de leitura é otimizado para consultas e pode ser replicado ou distribuído para suportar alta demanda.

Essa estratégia permite:

- baixa latência de leitura
- aumento da capacidade de resposta
- isolamento da carga de leitura

7.5 Crescimento do Ledger

O crescimento contínuo do ledger é tratado por meio de:

- particionamento temporal
- organização por períodos
- arquivamento de dados históricos

Essa abordagem reduz o impacto em performance e facilita a manutenção do sistema.

7.6 Isolamento de Carga

A separação entre escrita, processamento e leitura garante isolamento entre os diferentes tipos de carga.

Exemplos:

- picos de leitura não impactam o registro de lançamentos
- falhas no processamento não interrompem a escrita
- aumento de ingestão não afeta consultas

7.7 Estratégia de Evolução

A arquitetura permite evolução incremental da solução, incluindo:

- adoção de diferentes tecnologias de persistência
- criação de novos modelos de leitura
- inclusão de novos consumidores de eventos
- evolução de regras de negócio sem impacto no ledger

7.8 Resiliência e Recuperação

A solução garante resiliência por meio de:

- processamento assíncrono
- uso de Dead Letter Queue (DLQ) para falhas pontuais

- reprocessamento completo via ledger

Essa abordagem permite recuperação tanto em nível operacional quanto estrutural.

8. Fluxos e Responsabilidades da Solução

8.1 Visão Geral dos Fluxos

A solução é estruturada com base na separação entre escrita, processamento e leitura, garantindo desacoplamento entre os componentes.

Os principais fluxos são:

- registro de lançamentos financeiros
- processamento de eventos
- consulta de saldo
- tratamento de falhas
- reprocessamento

8.2 Fluxo de Registro de Lançamentos (Write)

1. O cliente (Web, Mobile ou integração) envia a requisição para a Transaction API
2. A Transaction API valida os dados da requisição
3. O lançamento é persistido no ledger (modelo append-only)
4. Um evento de transação é publicado no event stream
5. A operação é concluída sem dependência do processamento posterior

---

8.3 Fluxo de Consolidação de Saldo (Processamento)

1. O Balance Processor consome eventos do event stream
2. O evento é validado e processado
3. O saldo consolidado é calculado de forma incremental
4. O modelo de leitura é atualizado

8.4 Fluxo de Consulta de Saldo (Read)

1. O cliente envia requisição para a Query API
2. A Query API consulta o modelo de leitura
3. O saldo consolidado é retornado

8.5 Fluxo de Tratamento de Falhas

1. O Balance Processor tenta processar o evento
2. Em caso de falha, são realizadas tentativas de retry
3. Persistindo a falha, o evento é encaminhado para a Dead Letter Queue (DLQ)

A DLQ permite isolamento e tratamento posterior de eventos problemáticos.

8.6 Fluxo de Reprocessamento

A solução prevê dois tipos de reprocessamento:

8.6.1 Reprocessamento Pontual (DLQ)

- eventos são recuperados da DLQ
- após análise/correção, são reprocessados individualmente

8.6.2 Reprocessamento Completo (Ledger)

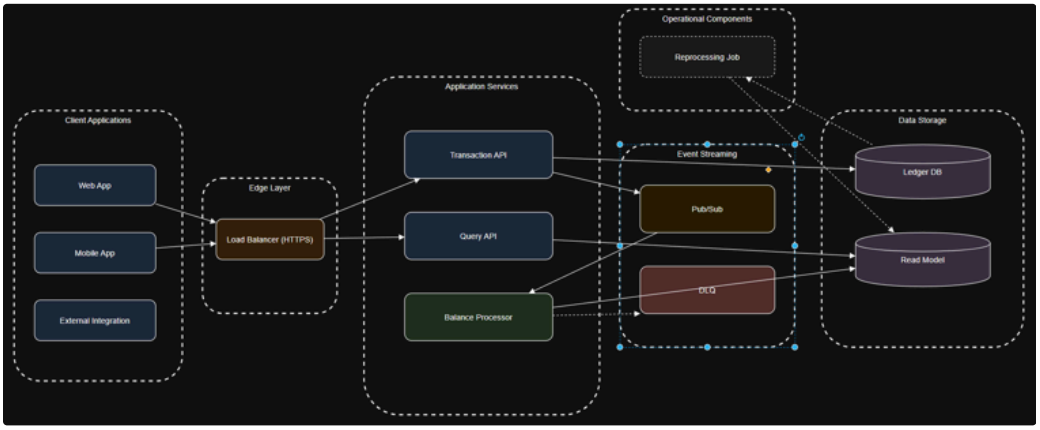
- o ledger é utilizado como fonte de verdade
- os registros são reprocessados em ordem
- o modelo de leitura é reconstruído

8.7 Responsabilidades dos Componentes

- **Transaction API:** responsável pelo registro dos lançamentos e publicação de eventos
- **Event Stream:** responsável pelo transporte e desacoplamento de eventos
- **Balance Processor:** responsável pelo processamento e consolidação do saldo
- **Query API:** responsável pela consulta do modelo de leitura
- **Ledger:** responsável pelo armazenamento imutável das transações
- **Read Model:** responsável pela leitura otimizada
- **DLQ:** responsável pelo isolamento de falhas

9. Arquitetura Técnica

Diagrama Arquitetural:



9.1 Visão Geral

A arquitetura técnica da solução foi definida com base nos requisitos de escalabilidade, resiliência e desacoplamento entre os componentes.

A solução adota uma abordagem baseada em serviços, comunicação assíncrona e separação entre modelo de escrita e leitura.

Os principais princípios adotados são:

- separação entre responsabilidades (CQRS)
- comunicação orientada a eventos
- escalabilidade horizontal
- isolamento entre componentes
- resiliência a falhas

9.2 Camadas da Arquitetura

A arquitetura é organizada nas seguintes camadas:

Camada de Cliente

Responsável pela interação com o usuário ou sistemas externos.

- Web App

- Mobile App
- Integrações externas

Camada de Entrada (API Layer)

Responsável por expor os serviços da aplicação diretamente aos clientes.

As responsabilidades de autenticação e controle de acesso são tratadas pelas próprias APIs.

- Transaction API (Write)
- Query API (Read)

Camada de Processamento

Responsável por expor os serviços da aplicação diretamente aos clientes.

As responsabilidades de autenticação e controle de acesso são tratadas pelas próprias APIs.

- Transaction API (Write)
- Query API (Read)

Camada de Dados

Responsável pelo armazenamento e disponibilização das informações.

- Ledger (write model)
- Read Model (read model)

Camada Operacional

Responsável pelo armazenamento e disponibilização das informações.

- Ledger (write model)
- Read Model (read model)

Camada de Segurança

Responsável por garantir proteção dos dados, controle de acesso e segurança das comunicações entre os componentes da solução.

Essa camada atua de forma transversal e contempla:

- autenticação de clientes
- autorização de acesso aos serviços
- proteção de APIs
- controle de acesso entre serviços
- criptografia de dados em trânsito e em repouso

9.3 Fluxo Técnico da Solução

O fluxo técnico segue o seguinte padrão:

Client → API → Ledger → Event Stream → Processor → Read Model

Fluxos adicionais:

Fluxo de falha

Processor → DLQ

Fluxo de recuperação

Ledger → Reprocessing → Read Model

9.4 Estratégia de Comunicação

A solução utiliza dois padrões de comunicação:

Comunicação síncrona

Utilizada entre cliente e APIs. baseada em HTTP/REST

Comunicação assíncrona

Utilizada entre os serviços internos. baseada em eventos permite desacoplamento permite escalabilidade

9.5 Isolamento de Componentes

Cada componente da arquitetura pode ser implantado e escalado de forma independente.

Isso permite:

- evitar falhas em cascata
- escalar apenas partes críticas
- evoluir componentes sem impacto global

9.6 Estratégia de Deploy

A solução pode ser implantada utilizando containers ou serviços gerenciados em nuvem.

Cada componente pode ser executado de forma independente, permitindo:

- deploy desacoplado
- atualização incremental
- rollback isolado

9.7 Considerações de Resiliência

A arquitetura foi projetada para lidar com falhas de forma controlada.

As principais estratégias adotadas são:

- uso de DLQ para isolamento de erros
- retries automáticos no processamento de eventos
- reprocessamento baseado no ledger
- separação entre fluxos críticos e operacionais

9.8 Diagramas de Sequência

Os diagramas de sequência são utilizados para representar o comportamento dinâmico da solução. Eles evidenciam a interação entre os componentes ao longo do tempo e complementam a visão estrutural da arquitetura.

9.8.1 Sequência — Registro de Lançamento

(descrever o fluxo)

9.8.2 Sequência — Processamento de Saldo

(descrever o fluxo)

9.8.3 Sequência — Reprocessamento

(descrever o fluxo)

9.9 Estratégia de Segurança

A arquitetura incorpora mecanismos de segurança em múltiplas camadas. Esses mecanismos garantem proteção dos dados e dos serviços ao longo de todo o fluxo da aplicação.

9.9.1 Segurança na Camada de Entrada

O acesso aos serviços é protegido por meio de HTTPS (TLS).

Isso garante criptografia em trânsito entre clientes e APIs.

As APIs são responsáveis por:

- validação de autenticação (ex: JWT)
- controle de acesso
- validação de requisições

9.9.2 Autenticação e Autorização

A solução utiliza autenticação baseada em tokens.

Isso permite:

- identificação segura dos clientes
- controle de acesso baseado em permissões
- isolamento entre tenants

9.9.3 Segurança na Comunicação entre Serviços

A comunicação entre os serviços é realizada por canais seguros. Além disso, são utilizados mecanismos de identidade para garantir que apenas componentes autorizados possam interagir entre si.

9.9.4 Proteção de Dados

Os dados são protegidos por meio de:

- criptografia em trânsito, garantindo segurança durante a comunicação
- criptografia em repouso, garantindo proteção dos dados armazenados
- controle de acesso, assegurando que apenas entidades autorizadas possam acessar os dados

9.9.5 Isolamento de Dados

A solução garante isolamento lógico entre os dados dos diferentes comerciantes. Esse isolamento é realizado por meio do uso de identificadores de tenant.

📄 10. Tecnologias e Ferramentas

10.1 Visão Geral

A escolha das tecnologias foi realizada com base nos requisitos de escalabilidade, resiliência, desacoplamento e simplicidade da solução.

A arquitetura prioriza o uso de serviços gerenciados, reduzindo a complexidade operacional e permitindo foco na lógica de negócio.

10.2 Camada de Aplicação

Tecnologia

- .NET / ASP.NET Core, an open-source web development framework | .NET Core
- Cloud Run (GCP)

Justificativa

A utilização do .NET permite alta produtividade no desenvolvimento e forte suporte a APIs modernas.

O Cloud Run possibilita execução de aplicações em containers com escalabilidade automática, reduzindo a necessidade de gerenciamento de infraestrutura.

Trade-offs

- (+) Escalabilidade automática
- (+) Redução de esforço operacional
- (-) Menor controle sobre infraestrutura

10.3 Mensageria

Tecnologia

- Google Cloud Pub/Sub

Justificativa

O Pub/Sub oferece comunicação assíncrona altamente escalável, permitindo desacoplamento entre os serviços e processamento distribuído.

É adequado para cenários event-driven e suporta alta taxa de throughput.

Trade-offs

- (+) Alta escalabilidade
- (+) Baixo acoplamento
- (-) Consistência eventual
- (-) Complexidade no tratamento de falhas

10.4 Processamento Assíncrono

Tecnologia

- Cloud Run (Workers)

Justificativa

O Balance Processor é executado como serviço independente, permitindo escalabilidade horizontal baseada no volume de eventos.

Essa abordagem garante flexibilidade e isolamento do processamento.

Trade-offs

- (+) Escala independente
- (+) Melhor uso de recursos
- (-) Complexidade de orquestração

10.5 Armazenamento de Dados

Ledger (Write Model)

Tecnologia

- Cloud SQL ou Bigtable

Justificativa

O ledger exige armazenamento confiável e consistente.

- Cloud SQL → melhor para consistência relacional
- Bigtable → melhor para alta escala e volume

A escolha pode variar conforme o volume esperado de dados.

Trade-offs

- Cloud SQL

  - (+) Consistência forte
  - (-) Escalabilidade limitada
- Bigtable
  - (+) Alta escalabilidade
  - (-) Maior complexidade

Read Model

Tecnologia

- Firestore ou BigQuery

Justificativa

O modelo de leitura é otimizado para consultas rápidas e agregações.

- Firestore → baixa latência (tempo real)
- BigQuery → análise e agregação em grande escala

Trade-offs

- Firestore
  - (+) Baixa latência
  - (-) Limitações analíticas
- BigQuery
  - (+) Alta capacidade analítica
  - (-) Latência maior

10.6 Dead Letter Queue (DLQ)

Tecnologia

- Pub/Sub Dead Letter Topic

Justificativa

Permite isolar eventos que falharam no processamento, evitando impacto no fluxo principal.

Trade-offs

- (+) Resiliência
- (+) Observabilidade de falhas
- (-) Necessidade de gestão operacional

10.7 Reprocessamento

Tecnologia

- Cloud Run Jobs

Justificativa

Permite execução sob demanda de processos de reconstrução do modelo de leitura a partir do ledger.

Trade-offs

- (+) Flexibilidade
- (+) Controle manual
- (-) Execução não contínua

10.8 Segurança

Tecnologias

- HTTPS (TLS)
- JWT (autenticação)
- IAM (controle entre serviços)

Justificativa

Essas tecnologias garantem proteção dos dados, controle de acesso e segurança na comunicação.

10.9 Observabilidade

Tecnologias

- Cloud Logging
- Cloud Monitoring

Justificativa

Permitem monitorar o comportamento do sistema, identificar falhas e acompanhar métricas de desempenho.

11. Segurança

11.1 Visão Geral

A segurança da solução é tratada de forma transversal, abrangendo autenticação, autorização, proteção de dados e controle de acesso entre os componentes.

11.2 Autenticação

A autenticação é realizada por meio de tokens (JWT), garantindo a identificação segura dos clientes.

11.3 Autorização

O acesso aos recursos é controlado com base em permissões, garantindo que cada cliente acesse apenas seus próprios dados.

11.4 Segurança na Comunicação

Toda comunicação é realizada via HTTPS (TLS), garantindo criptografia em trânsito.

11.5 Proteção de Dados

Os dados são protegidos por meio de:

- criptografia em trânsito
- criptografia em repouso
- controle de acesso

11.6 Isolamento por Tenant

Os dados são segregados logicamente por identificador de tenant, evitando acesso indevido entre clientes.

11.7 Segurança entre Serviços

A comunicação entre serviços é protegida por identidade e controle de acesso, garantindo que apenas componentes autorizados possam interagir.

📄 12. Observabilidade

12.1 Visão Geral

A observabilidade da solução é responsável por permitir o monitoramento, diagnóstico e análise do comportamento do sistema em tempo de execução.

A arquitetura adota práticas que possibilitam visibilidade completa dos fluxos, identificação de falhas e acompanhamento de métricas de desempenho.

12.2 Logs

Os serviços da aplicação geram logs estruturados, permitindo análise e rastreabilidade das operações.

Características:

- logs estruturados (JSON)
- identificação de requisições (correlation id)
- registro de erros e eventos relevantes

Objetivos:

- facilitar debugging
- permitir auditoria de eventos
- apoiar análise de falhas

12.3 Métricas

A solução coleta métricas operacionais que permitem acompanhar a saúde do sistema.

Principais métricas:

- latência das APIs
- throughput de eventos
- taxa de erros
- tempo de processamento

Essas métricas permitem identificar degradação de performance e gargalos.

12.4 Monitoramento

O monitoramento é responsável por acompanhar o comportamento do sistema em tempo real.

Inclui:

- dashboards com métricas de negócio e técnicas
- acompanhamento de disponibilidade dos serviços
- visualização de filas e processamento de eventos

12.5 Alertas

A solução define alertas para cenários críticos, permitindo ação rápida em caso de falhas.

Exemplos:

- aumento da taxa de erro
- crescimento anormal da DLQ
- latência elevada
- falha no processamento de eventos

12.6 Tracing Distribuído

A arquitetura permite rastrear o fluxo de uma requisição entre múltiplos serviços.

Isso possibilita:

- identificar gargalos
- entender dependências
- analisar o tempo de resposta ponta a ponta

12.7 Observabilidade do Fluxo de Eventos

Considerando o uso de mensageria, a observabilidade inclui:

- monitoramento do event stream
- acompanhamento do consumo de eventos
- análise de falhas de processamento

12.8 Observabilidade da DLQ

A DLQ é monitorada como indicador de falhas operacionais.

A solução permite:

- identificar eventos não processados
- analisar causas de falha
- acionar processos de reprocessamento

12.9 Ferramentas

A solução pode utilizar ferramentas gerenciadas para observabilidade, tais como:

- Cloud Logging
- Cloud Monitoring
- Cloud Trace

12.10 Evolução da Observabilidade

A solução pode evoluir para o uso de ferramentas especializadas de observabilidade, como Datadog, conforme o aumento da complexidade e necessidade de análise avançada.

Nesse cenário, os dados coletados pelo ambiente de nuvem podem ser integrados a uma plataforma externa, permitindo:

- correlação avançada entre logs, métricas e traces
- dashboards mais completos
- maior eficiência no troubleshooting

A arquitetura foi desenhada de forma a não acoplar a solução a uma ferramenta específica de observabilidade, permitindo flexibilidade na evolução tecnológica.

Custos(FinOps)

13. Custos (FinOps)

13.1 Premissas da Estimativa

Para análise de custo, foi considerado um cenário inicial simplificado com as seguintes premissas:

- 10 usuários ativos
- operação em horário comercial (8 horas por dia)
- volume moderado de transações
- baixa concorrência simultânea
- ambiente com apenas uma instância lógica por serviço (escala mínima)

Essa abordagem permite estabelecer uma base de referência para evolução dos custos conforme o crescimento da solução.

13.2 Componentes da Arquitetura Avaliados

A estimativa considera os seguintes componentes:

- APIs (Cloud Run)
- Balance Processor
- Pub/Sub
- Armazenamento (Ledger e Read Model)
- Observabilidade

13.3 Estimativa de Consumo (Cenário Base)

Camada de Aplicação

- baixo volume de requisições
- instâncias sob demanda (serverless)
- consumo proporcional ao uso

👉 custo tende a ser **baixo**, devido ao modelo pay-per-use

Mensageria (Pub/Sub)

- baixo volume de eventos por usuário
- processamento contínuo durante horário comercial

👉 custo tende a ser **baixo a moderado**, dependendo da frequência de eventos

---

Processamento (Balance Processor)

- ativado conforme chegada de eventos
- execução leve por transação

👉 custo tende a ser **baixo**

Armazenamento

- crescimento contínuo do ledger
- read model atualizado incrementalmente

👉 custo tende a ser **baixo inicialmente**, com crescimento gradual ao longo do tempo

---

Observabilidade

- geração de logs e métricas
- baixo volume no cenário inicial

👉 custo tende a ser **baixo**, podendo crescer conforme aumento de logs

---

13.4 Comportamento com Escala

A arquitetura foi projetada para que os custos cresçam de forma proporcional ao uso.

Crescimento esperado:

- aumento de usuários → aumento de requisições
- aumento de requisições → aumento de eventos
- aumento de eventos → maior processamento e armazenamento

13.5 Projeção de Crescimento e Pontos de Ajuste

A arquitetura foi projetada para escalar de forma proporcional ao volume de uso. No entanto, determinados pontos podem exigir ajustes conforme o crescimento da carga.

Até ~50 req/s (cenário atual)

- operação com serviços serverless
- escalabilidade automática suficiente
- custo proporcional ao uso
- sem necessidade de otimizações estruturais

Entre 50 e 200 req/s

Possíveis ajustes:

- otimização do consumo de recursos nas APIs
- ajuste de concorrência no Cloud Run
- controle mais rigoroso de logs

Motivação:

- aumento do custo por volume de execução
- maior pressão sobre processamento assíncrono

Acima de 200 req/s

Possíveis ajustes:

- introdução de cache para reduzir leituras no read model
- particionamento mais agressivo do ledger
- otimização do processamento de eventos

Motivação:

- aumento significativo de volume de dados
- maior custo de leitura e processamento

Cenários de alto volume (>500 req/s)

Possíveis ajustes estruturais:

- revisão da estratégia de armazenamento
- adoção de soluções mais otimizadas para leitura (ex: sistemas analíticos)
- otimização da arquitetura de eventos

Motivação:

- necessidade de eficiência em larga escala
- controle de custo em cenários de alta carga

13.6 Estratégias de Controle de Custos

A solução adota práticas para evitar crescimento descontrolado:

- uso de serviços serverless (cobrança por uso)
- controle de retenção de logs
- separação entre dados quentes e frios
- processamento assíncrono para otimização de recursos

13.7 Evolução da Estratégia de Custos

Conforme o crescimento da solução, podem ser adotadas estratégias adicionais:

- otimização de queries
- uso de cache para reduzir leituras
- arquivamento de dados históricos
- ajuste fino de observabilidade

13.8 Estimativa de Custo (Cenário Base)

Considerando uma carga de até 50 requisições por segundo, operando em horário comercial, a solução apresenta a seguinte estimativa de custo mensal:

- Camada de aplicação (Cloud Run): $50 a $150
- Mensageria (Pub/Sub): $10 a $40
- Processamento (Balance Processor): $20 a $100
- Armazenamento: $30 a $150
- Observabilidade: $10 a $50

Total estimado: $120 a $400 por mês